



False positives vs. real vulnerabilities in WebGoat

How a runtime (IAST) approach reports real flaws and stays silent on the noise guess-based scanners produce.

A legacy SAST/DAST scanner infers vulnerabilities from source patterns or HTTP responses, so it cannot tell whether the dangerous code actually ran. WebGoat is full of *simulated* lessons that only check your input against a pattern and print “Lesson Passed!”. Contrast Assess instruments the running code and reports a vulnerability only when untrusted data reaches a dangerous operation during execution. So it reports WebGoat’s real flaws and ignores the simulated ones, with no manual triage. The best way to handle a false positive is to never generate it.

Simulated lessons — a scanner flags these, Contrast does not

Endpoint	What the code actually does	Contrast
POST /SqlInjectionMitigations/attack10b	Runs a regex over the text you submit to check your answer. No database call.	No finding
POST /SSRF/task1	Compares your url to fixed strings. Never opens a connection.	No finding
GET /CrossSiteScripting/attack5a	Succeeds on String.contains("<script>"). Not a live HTML sink.	No finding

Real vulnerabilities — Contrast reports these from normal use

Endpoint	Vulnerability	The sink that executes	Contrast
POST /SqlInjection/attack2	SQL Injection	Statement.executeQuery(userQuery)	Reports
POST /SqlInjection/assignment5a	SQL Injection	input concatenated into last_name = '...'	Reports
POST /SqlInjection/attack8	SQL Injection	name + auth_tan concatenated into query	Reports
POST /SqlInjectionAdvanced/attack6a	SQL Injection	last name concatenated into query	Reports
POST /SqlInjection/assignment5b	SQL Injection	userid concatenated (login_count is bound)	Reports
GET /SqlInjectionMitigations/servers	SQL Injection	column concatenated into ORDER BY	Reports
POST /xxe/simple	XML External Entity	XML body parsed with external entities	Reports
POST /InsecureDeserialization/task	Untrusted deserialization	ObjectInputStream.readObject() on request	Reports
POST /VulnerableComponents/attack1	Vulnerable component	XStream.fromXML(input)	Reports
GET /crypto/hashing/md5	Weak cryptography	MessageDigest.getInstance("MD5")	Reports
POST /SSRF/task2	SSRF	new URL(input).openStream() server-side	Reports

Safe by design — a true negative, not a miss

Endpoint	Why it is safe	Contrast
POST /SqlInjectionMitigation/s/attack12a	Fully parameterized PreparedStatement; every value bound with setString.	No finding

Why it matters for the evaluation

A false-positive count only means something relative to a tool that produces false positives. A guess-based scanner flags the simulated lessons above and the burden lands on your team to investigate and dismiss each one. Contrast reports the real flaws while the safe code stays quiet, so the triage queue starts smaller and cleaner. Every row here can be confirmed against a live run: deploy, exercise the app with normal input, and compare the Contrast findings to these tables.

It's never black and white.

Internal: external-facing collateral. Verify findings against the live Contrast instance and current product behavior, and route through review before sharing with a prospect. Endpoints reflect OWASP WebGoat v2025.3.